

CuTe DSL Overview

cute-tutorial

July 21, 2026

What is CuTe DSL?

CuTe DSL is a **Python** decorator-based DSL for writing hardware-aware **GPU kernels** with JIT compilation.

- Evolved from C++ CUTLASS / CuTe
- Zero-cost abstractions with low-level hardware control
- DLPack interop, native types, and JIT caching

Outline — Three Modules

1 Core Foundations

Python → GPU code, layouts & tensors

2 SIMT Programming Model

threads move data and do scalar work

3 TMA and MMA Programming

async tile movement, tensor-core math, and synchronization

Module 1

Core Foundations

Python → GPU code, layouts & tensors

Core Foundations Roadmap

- **Entry points:** `@cute.jit` for host-side JIT logic, `@cute.kernel` for GPU kernels.
- **Tracing model:** the same Python code builds IR first, then compiled code runs later.
- **Dynamic vs. constant arguments:** runtime values stay as IR arguments; `Constexpr` values specialize the generated code.
- **JIT cache and executors:** direct calls trace every time; `cute.compile` returns a reusable executor.
- **Tensor layouts:** CuTe describes tensors with `(Shape):(Stride)` and supports static or dynamic layout fields.

Two Decorators: @cute.jit and @cute.kernel

Two entry points:

- @cute.jit: host-side JIT functions
- @cute.kernel: GPU kernels

The First Example

```
import cutlass
import cutlass.cute as cute

@cute.jit
def add_dynamicexpr(b: cutlass.Float32):
    a: cutlass.Float32 = 2.0
    result = a + b
    print("[meta-stage] result =", result)
    cute.printf("[object-stage] result = %f\n", result)

add_dynamicexpr(5.0)
```

General Mechanism: Same Python Code Runs Twice

The same function body is used in two different contexts:
first to **build IR**, then to **run the compiled program**.

- ① **Pre-stage:** CuTe DSL rewrites the Python AST to capture structure.
- ② **Meta-stage:** Python runs on the host with proxy arguments; tensor operations emit MLIR.
- ③ **Object-stage:** MLIR is lowered and compiled; emitted operations run at runtime.
 - `print()` runs during tracing.
 - `cute.printf()` does not print during tracing; it is emitted as `cute.print`.

What Happens When We Call `add_dynamicexpr(5.0)`?

`add_dynamicexpr(5.0)` → **rewrite** → **trace** → **lower + compile**

❶ **Pre-stage: rewrite the Python AST**

Capture control-flow structure. For this straight-line example, there is no branch or loop to preserve.

❷ **Meta-stage: run Python with proxy values (tracing)**

The function body executes on the host; tensor operations append MLIR to the current block.

❸ **Object-stage: lower and compile MLIR**

The completed IR is lowered toward PTX/SASS; emitted operations run later.

The next slides zoom into the **meta-stage** one statement at a time.

Tracing Step by Step: add_dynamicexpr(5.0)

Tracing code

```
@cute.jit
def add_dynamicexpr(b: cutlass.Float32):
    a: cutlass.Float32 = 2.0
    result = a + b
    print("[meta-stage] result =", result)
    cute.printf("[object-stage] result = %f\n",
                result)
```

MLIR

```
// no IR operation yet
```

Terminal

```
<no output yet>
```

Tracing Step by Step: add_dynamicexpr(5.0)

Tracing code

```
@cute.jit
def add_dynamicexpr(b: cutlass.Float32):
    a: cutlass.Float32 = 2.0
    result = a + b
    print("[meta-stage] result =", result)
    cute.printf("[object-stage] result = %f\n",
                result)
```

MLIR

```
%cst = arith.constant 2.000000e+00 : f32
%0 = arith.addf %cst, %arg0 : f32
```

Terminal

```
<no output yet>
```

Tracing Step by Step: add_dynamicexpr(5.0)

Tracing code

```
@cute.jit
def add_dynamicexpr(b: cutlass.Float32):
    a: cutlass.Float32 = 2.0
    result = a + b
    print("[meta-stage] result =", result)
    cute.printf("[object-stage] result = %f\n",
                result)
```

MLIR

```
%cst = arith.constant 2.000000e+00 : f32
%0 = arith.addf %cst, %arg0 : f32
```

Terminal

```
[meta-stage] result = ?
```

Tracing Step by Step: add_dynamicexpr(5.0)

Tracing code

```
@cute.jit
def add_dynamicexpr(b: cutlass.Float32):
    a: cutlass.Float32 = 2.0
    result = a + b
    print("[meta-stage] result =", result)
    cute.printf("[object-stage] result = %f\n",
                result)
```

MLIR

```
%cst = arith.constant 2.000000e+00 : f32
%0 = arith.addf %cst, %arg0 : f32
cute.print("[object-stage] result = %f\0A\0A", %0) : f32
```

Terminal

```
[meta-stage] result = ?
```

No new trace-time output: `cute.printf` was emitted as `cute.print`.

Tracing Step by Step: add_constexpr(5.0)

Tracing code

```
@cute.jit
def add_constexpr(b: cutlass.Constexpr):
    a: cutlass.Float32 = 2.0
    result = a + b
    print("[meta-stage] result =", result)
    cute.printf("[object-stage] result = %f\n",
                result)
```

MLIR

```
// no IR operation yet
```

Terminal

```
[meta-stage] result = 7.0
```

Both inputs are known at trace time; a is explicitly typed as Float32.

Tracing Step by Step: add_constexpr(5.0)

Tracing code

```
@cute.jit
def add_constexpr(b: cutlass.Constexpr):
    a: cutlass.Float32 = 2.0
    result = a + b
    print("[meta-stage] result =", result)
    cute.printf("[object-stage] result = %f\n",
                result)
```

MLIR

```
%cst = arith.constant 7.000000e+00 : f32
cute.print("[object-stage] result = %f\0A\0A", %cst) : f32
```

Terminal

```
[meta-stage] result = 7.0
```

The generated IR has no runtime %arg0; the value was baked in.

JIT Cache: Dynamic vs. Constexpr Values

Dynamic parameter

```
add_dynamicexpr(5.0)
add_dynamicexpr(8.0)
```

Generated IR shape

```
%cst = arith.constant 2.000000e+00 : f32
%0 = arith.addf %cst, %arg0 : f32
cute.print(..., %0) : f32
```

The argument stays runtime data. New values with the same signature reuse the same native code because the IR shape is unchanged.

Constexpr parameter

```
add_constexpr(5.0) # result = 7.0
add_constexpr(8.0) # result = 10.0
```

Generated IR changes

```
// b = 5.0
%cst = arith.constant 7.000000e+00 : f32
cute.print(..., %cst) : f32

// b = 8.0
%cst = arith.constant 1.000000e+01 : f32
cute.print(..., %cst) : f32
```

The value is part of specialization. Changing it changes the IR hash, so the JIT cache must generate new native code.

Cache lookup uses the emitted MLIR hash, not just Python source.

Control Flow: The Rewrite Becomes Visible

Source shape

```
@cute.jit
def choose_runtime(x: cutlass.Int32):
    if x > cutlass.Int32(0):
        print("[trace] building then branch")
        cute.printf("[runtime] positive\n")
    else:
        print("[trace] building else branch")
        cute.printf("[runtime] non-positive\n")
```

Conceptual rewrite

```
pred = x > cutlass.Int32(0)

if_generate(
    pred,
    then_branch,
    else_branch,
)
```

Straight-line code barely uses the pre-stage. Runtime control flow needs it: branch bodies become regions that can be traced into IR.

Runtime Branching: if $x > 0$

Tracing code

```
@cute.jit
def choose_runtime(x: cutlass.Int32):
    if x > cutlass.Int32(0):
        print("[trace] building then branch")
        cute.printf("[runtime] positive\n")
    else:
        print("[trace] building else branch")
        cute.printf("[runtime] non-positive\n")
```

MLIR

```
%c0_i32 = arith.constant 0 : i32
%0 = arith.cmpi sgt, %arg0, %c0_i32 : i32
scf.if %0 {
  cute.print("[runtime] positive\0A\0A")
} else {
  cute.print("[runtime] non-positive\0A\0A")
}
```

Terminal

```
[trace] building then branch
[trace] building else branch
```

Both branch regions are traced; runtime chooses one branch.

Constexpr Value, Plain if: Still scf.if

Tracing code

```
@cute.jit
def choose_path(x: cutlass.Int32,
               use_fast: cutlass.Constexpr):
    if use_fast:
        print("[trace] building fast branch")
        cute.printf("[runtime] y = %d\n",
                    x + cutlass.Int32(1))
    else:
        print("[trace] building slow branch")
        cute.printf("[runtime] y = %d\n",
                    x + cutlass.Int32(100))
```

MLIR

```
%true = arith.constant true
scf.if %true {
  %c1_i32 = arith.constant 1 : i32
  %0 = arith.addi %arg0, %c1_i32 : i32
  cute.print("[runtime] y = %d\0A\0A", %0) : i32
} else {
  %c100_i32 = arith.constant 100 : i32
  %0 = arith.addi %arg0, %c100_i32 : i32
  cute.print("[runtime] y = %d\0A\0A", %0) : i32
}
```

Terminal

```
[trace] building fast branch
[trace] building slow branch
```

Constexpr alone does not make this a trace-time branch; plain if still uses the control-flow rewrite.

Trace-Time Branching: `if cutlass.const_expr(...)`

Tracing code

```
@cute.jit
def choose_path(x: cutlass.Int32,
               use_fast: cutlass.Constexpr):
    if cutlass.const_expr(use_fast):
        print("[trace] building fast branch")
        cute.printf("[runtime] y = %d\n",
                    x + cutlass.Int32(1))
    else:
        print("[trace] building slow branch")
        cute.printf("[runtime] y = %d\n",
                    x + cutlass.Int32(100))
```

MLIR

```
%c1_i32 = arith.constant 1 : i32
%0 = arith.addi %arg0, %c1_i32 : i32
cute.print("[runtime] y = %d\0A\0A", %0) : i32
```

Terminal

```
[trace] building fast branch
```

`cutlass.const_expr(...)` is the explicit signal to skip the control-flow rewrite. The predicate must be compile-time known; otherwise tracing errors.

Compile-Time Metaprogramming: Optional ReLU

Kernel sketch

```
@cute.kernel
def gemm(...,
        do_relu: cutlass.Constexpr):
    # main GEMM work
    ...

    if cutlass.const_expr(do_relu):
        # ReLU epilogue is emitted only
        # when do_relu is True
        ...
```

Two specializations

```
gemm(..., False)

// generated code:
main GEMM
store accumulator

gemm(..., True)

// generated code:
main GEMM
relu accumulator
store accumulator
```

The flag is a constant argument. It selects code during tracing, so the final kernel has no runtime branch for this option.

JIT Gotcha: Direct @cute.jit Calls

Code

```
import cutlass
import cutlass.cute as cute

@cute.jit
def hello():
    print("trace: hello")
    cute.printf("runtime: hello\n")

hello()
hello()
```

Question

How many times does each line print?

- trace: hello
- runtime: hello

Implicit Cache: Trace Every Call, Compile on Miss

First direct call

```
hello()  
  
trace Python  
emit MLIR  
cache lookup: miss  
compile MLIR -> native code  
run native code
```

Second direct call

```
hello()  
  
trace Python  
emit MLIR  
cache lookup: hit  
reuse compiled executor  
run native code
```

Because generated MLIR bytecode is part of the implicit cache key, CuTe must trace before it can decide whether compilation is a cache hit.

Use `cute.compile` to Reuse a JIT Executor

Code

```
import cutlass
import cutlass.cute as cute

@cute.jit
def hello():
    print("trace: hello")
    cute.printf("runtime: hello\n")

compiled_func = cute.compile(hello)

compiled_func()
compiled_func()
```

Terminal

```
trace: hello
runtime: hello
runtime: hello
```

In this pattern, `cute.compile` traces and compiles once, returning a JIT Executor. Reusing that executor skips Python tracing; calling `cute.compile` again would compile again.

Exercise: Constexpr and cute.compile

Code

```
import cutlass
import cutlass.cute as cute

@cute.jit
def add(a, b, print_result: cutlass.Constexpr):
    if print_result:
        cute.printf("Result: %d\n", a + b)
    return a + b

jit_executor = cute.compile(add, 1, 2, True)
jit_executor(1, 2)
```

Questions

What prints during
`cute.compile(...)`?

What prints during `jit_executor(1, 2)`?

Does the emitted MLIR keep control flow, or is it already specialized?

`print_result` is a compile-time argument, so it is not passed to the executor at runtime.

CuTe Tensor Layout: (Shape):(Stride)

Layout notation

```
(3):(1)  
(2,2):(2,1)  
(?,?):(?{i64},1)
```

Row-major example

```
layout = (M,N):(N,1)  
  
offset(i,j) = i * N + j
```

A layout maps logical coordinates to offsets.

Shape: logical extents Stride: offset step per mode ?: runtime-known dimension or stride

Argument Annotations: What Gets Inferred?

Unannotated scalars

```
@cute.jit
def foo(x):
    cute.printf("x = {}\n", x)

foo(1.0)  // x: Float32, MLIR f32
foo(1)    // x: Int32,   MLIR i32
foo(True) // x: Boolean, MLIR i1
```

By default, arguments are dynamic and CuTe infers their DSL types from the call-site values.

Annotate when intent matters

```
def foo(x: cutlass.Float64):
    ...

def bar(n: cutlass.Int64):
    ...

def baz(flag: cutlass.Constexpr):
    ...

def qux(tensor: cute.Tensor):
    ...
```

Use annotations for precision or width, compile-time constants, and type safety. For tensors, static vs. dynamic layout is decided by the argument object passed at the call site.

Static Layout: Wrap with from_dlpack

Host code

```
import cutlass.cute as cute
from cutlass.cute.runtime import from_dlpack

@cute.jit
def foo(tensor):
    print(tensor.layout)
    cute.printf("tensor: {}", tensor)

a = torch.tensor([1, 2, 3],
                  dtype=torch.uint16)
a_pack = from_dlpack(a)

compiled = cute.compile(foo, a_pack)
compiled(a_pack)
```

Trace-time output

```
(3):(1)
```

Runtime output

```
tensor: ... o (3):(1) =
( 1, 2, 3 )
```

`print(tensor.layout)` runs while tracing.
`cute.printf(...)` is emitted and prints when the compiled executor runs.

Dynamic Layout: Pass torch.Tensor Directly

Host code

```
a = torch.tensor([[1, 2], [3, 4]],
                  dtype=torch.uint16)
compiled = cute.compile(foo, a)
compiled(a)

b = torch.tensor([[11, 12],
                  [13, 14],
                  [15, 16]],
                  dtype=torch.uint16)
compiled(b)
```

Trace-time layout

```
(?,?):(?{i64},1)
```

Runtime layouts

```
tensor: ... o (2,2):(2,1)
tensor: ... o (3,2):(2,1)
```

The generated code keeps selected layout fields dynamic, so one compiled function can handle multiple compatible runtime shapes.

Module 2

SIMT Programming Model

threads, CTAs, scalar work, and tile ownership

- **Vector add:** first `@cute.kernel`; one thread owns one element.
- **Reductions:** threads cooperate on scalar work.
- **2D indexing:** map a flat thread id to (row, col) tensor coordinates.
- **CTA tile views:** use `cute.local_tile` so each CTA owns a rectangular tile.
- **Shared-memory staging:** introduce `make_layout`, `SmemAllocator`, and CTA barriers.

Vector Add: Familiar CUDA-Style SIMT

GPU kernel

```
@cute.kernel
def vector_add_kernel(a: cute.Tensor,
                     b: cute.Tensor,
                     c: cute.Tensor):
    tid, _, _ = cute.arch.thread_idx()
    bidx, _, _ = cute.arch.block_idx()
    bdim, _, _ = cute.arch.block_dim()

    idx = bidx * bdim + tid
    n = a.shape[0]

    if idx < n:
        c[idx] = a[idx] + b[idx]
```

Host JIT launcher

```
@cute.jit
def vector_add(a: cute.Tensor,
              b: cute.Tensor,
              c: cute.Tensor):
    n = a.shape[0]
    grid = cute.ceil_div(n, THREADS_PER_BLOCK)
    vector_add_kernel(a, b, c).launch(
        grid=(grid, 1, 1),
        block=(THREADS_PER_BLOCK, 1, 1),
    )
```

One thread owns one element. `cute.arch.*` gives CUDA-style indices; tensor indexing expresses global-memory loads and stores.

Reduction: Shared Memory + One Atomic

GPU kernel

```
@cute.kernel
def reduce_kernel(src, out, smem_layout):
    tid, _, _ = cute.arch.thread_idx()
    bidx, _, _ = cute.arch.block_idx()
    idx = bidx * BLOCK + tid

    buf = SmemAllocator().allocate_tensor(
        cutlass.Float32, smem_layout)

    buf[tid] = cutlass.Float32(0.0)
    if idx < src.shape[0]:
        buf[tid] = src[idx]
    cute.arch.barrier()

    stride = BLOCK // 2
    while stride > 0:
        if tid < stride:
            buf[tid] += buf[tid + stride]
        cute.arch.barrier()
        stride = stride // 2

    if tid == 0:
        cute.arch.atomic_add(out.iterator, buf[0])
```

Host JIT launcher

```
@cute.jit
def reduce_sum(src: cute.Tensor,
               out: cute.Tensor):
    n = src.shape[0]
    grid = cute.ceil_div(n, BLOCK)
    smem_layout = cute.make_layout(BLOCK)

    reduce_kernel(src, out, smem_layout).launch(
        grid=(grid, 1, 1),
        block=(BLOCK, 1, 1),
    )
```

The host initializes `out[0] = 0`. Each CTA reduces into shared memory; one thread atomically adds the CTA sum to the global result.

cute.make_layout: Shape + Stride

Signature

```
cute.make_layout(shape, *, stride=None)
```

- shape: integer or tuple of mode sizes.
- stride: optional keyword-only stride.
- returns a `cute.Layout`.

Common uses

```
// 1D compact scratch
smem_layout = cute.make_layout(BLOCK)

// 2D row-major tile
tile_layout = cute.make_layout(
    (TILE_M, TILE_N),
    stride=(TILE_N, 1),
)

smem = SmemAllocator()
buf = smem.allocate_tensor(
    cutlass.Float32, smem_layout)
```

If `stride` is omitted, CuTe computes a default "left-most" layout. Pass `stride=` when the physical memory order matters.

API doc: https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl_api/cute.html#cutlass.cute.make_layout

2D Indexing: Flat Thread Id to (row, col)

GPU kernel

```
@cute.kernel
def copy_2d_kernel(src: cute.Tensor,
                   dst: cute.Tensor):
    tidx, _, _ = cute.arch.thread_idx()
    bidx, _, _ = cute.arch.block_idx()
    bdim, _, _ = cute.arch.block_dim()

    linear = bidx * bdim + tidx
    rows, cols = src.shape
    numel = rows * cols

    if linear < numel:
        row = linear // cols
        col = linear % cols
        dst[row, col] = src[row, col]
```

Host JIT launcher

```
@cute.jit
def copy_2d(src: cute.Tensor,
            dst: cute.Tensor):
    rows, cols = src.shape
    n = rows * cols
    grid = cute.ceil_div(
        n, THREADS_PER_BLOCK)

    copy_2d_kernel(src, dst).launch(
        grid=(grid, 1, 1),
        block=(THREADS_PER_BLOCK, 1, 1),
    )
```

Same SIMT skeleton as vector add. The only new step is decoding one flat element id into tensor coordinates.

CTA Tile Views with `cute.local_tile`

GPU kernel

```
@cute.kernel
def local_tile_copy_kernel(src: cute.Tensor,
                           dst: cute.Tensor):
    tid_x, _, _ = cute.arch.thread_idx()
    block_m, block_n, _ = cute.arch.block_idx()
    block_dim, _, _ = cute.arch.block_dim()

    src_tile = cute.local_tile(
        src, (TILE_M, TILE_N), (block_m, block_n))
    dst_tile = cute.local_tile(
        dst, (TILE_M, TILE_N), (block_m, block_n))

    i = tid_x
    while i < TILE_M * TILE_N:
        row = i // TILE_N
        col = i % TILE_N
        dst_tile[row, col] = src_tile[row, col]
        i += block_dim
```

Host JIT launcher

```
@cute.jit
def local_tile_copy(src: cute.Tensor,
                    dst: cute.Tensor):
    rows, cols = src.shape
    grid_m = cute.ceil_div(rows, TILE_M)
    grid_n = cute.ceil_div(cols, TILE_N)

    local_tile_copy_kernel(src, dst).launch(
        grid=(grid_m, grid_n, 1),
        block=(THREADS_PER_CTA, 1, 1),
    )
```

`local_tile` returns a tensor view; it does not copy data.

Module 3

TMA and MMA Programming

async tile movement, tensor-core math, and synchronization

CuTe DSL Namespaces

```
import cutlass
import cutlass.cute as cute
from cutlass.cute.nvgpu import cpasync
```

- `cutlass.*`: native DSL types and compile-time markers, such as `Float32`, `BFloat16`, and `Constexpr`.
- `cutlass.cute.*`: core CuTe abstractions and generic algorithms, such as `make_layout`, `local_tile`, `make_copy_atom`, and `copy`.
- `cutlass.cute.nvgpu.*`: NVIDIA GPU-specific operations and helpers, such as `cpasync.CopyG2S0p`, `CopyBulkTensorTileG2S0p`, and `tma_partition`.

The namespace split is a layering signal: generic composition lives in `cute.*`; backend-specific operation kinds live under `cute.nvgpu.*`. The current public CuTe DSL backend targets NVIDIA CUDA/PTX/SASS.

Before the First TMA Load

TMA reuses the machinery we already introduced:

- CTA tile ownership with `cute.local_tile`
- explicit SMEM layout with `cute.make_layout`
- shared-memory allocation with `SmemAllocator`

It adds three new pieces:

- **TMA atom:** describes the GMEM-to-SMEM tile copy operation.
- **Partitioned views:** `group_modes` folds tensor modes; `cpasync.tma_partition` derives copy views.
- **mbarrier:** the kernel waits until the async TMA transaction completes.

A CuTe atom is a hardware primitive plus CuTe metadata.

Copy atoms

- describe low-level data movement
- examples: SIMT copy, async copy, TMA copy
- consumed by `cute.copy`

MMA atoms

- describe tensor-core math instructions
- encode instruction shape and operand metadata
- consumed by MMA algorithms such as `cute.gemm`

This is unrelated to `atomic_add`; “atom” means the small hardware primitive CuTe builds larger tiled operations from.

Copy Atom Call Sequence

TMA bulk tensor copy

```
tma_op = cpasync.CopyBulkTensorTileG2S0p()

tma_atom, src_tma = cpasync.make_tiled_tma_atom(
    tma_op,
    src,
    smem_layout,
    (TILE_M, TILE_K),
)

# after tma_partition(...)
cute.copy(
    tma_atom,
    gmem_view[(None, block_m, block_k)],
    smem_view,
    tma_bar_ptr=tma_barrier,
)
```

TMA uses a specialized helper to build descriptor-style metadata and a TMA-compatible source tensor.

***Op()** selects the operation kind; it does not issue the instruction. **cute.copy(...)** is where the data movement is issued.

Non-TMA cp.async

```
cp_op = cpasync.CopyG2S0p()

cp_atom = cute.make_copy_atom(
    cp_op,
    cutlass.Float32,
    num_bits_per_copy=128,
)

# after choosing tensor views
cute.copy(
    cp_atom,
    gmem_view,
    smem_view,
)
```

Generic copy atoms are built with `cute.make_copy_atom`; tensors are supplied later when issuing `cute.copy`.

TMA Load: Which Call Does What?

Trace-time setup

```
op = cpasync.CopyBulkTensorTileG2S0p()

tma_atom, src_tma = cpasync.make_tiled_tma_atom(
    op,
    src,
    smem_layout,
    (TILE_M, TILE_K),
)
```

`CopyBulkTensorTileG2S0p()` chooses a GMEM-to-SMEM TMA copy operation. `make_tiled_tma_atom(...)` builds the CuTe atom and TMA-compatible source tensor. It does not issue the load.

Kernel-side issue

```
cute.copy(
    tma_atom,
    gmem_view[(None, block_m, block_k)],
    smem_view,
    tma_bar_ptr=tma_barrier,
)
```

`cute.copy(...)` is the call that issues the **async TMA load**. The `mbarrier` is used to wait for that transaction before consuming SMEM.

Minimal TMA Load: JIT Setup

Build the TMA atom

```
@cute.jit
def launch_tma_load(src: cute.Tensor,
                    out: cute.Tensor):
    smem_layout = cute.make_layout(
        (TILE_M, TILE_K),
        stride=(TILE_K, 1))

    tma_atom, src_tma = cpasync.make_tiled_tma_atom(
        cpasync.CopyBulkTensorTileG2S0p(),
        src,
        smem_layout,
        (TILE_M, TILE_K),
    )
```

Launch one CTA per tile

```
grid_m = cute.ceil_div(
    src.layout.shape[0], TILE_M)
grid_k = cute.ceil_div(
    src.layout.shape[1], TILE_K)

tma_load_kernel(
    tma_atom,
    src_tma,
    out,
    smem_layout,
).launch(
    grid=(grid_m, grid_k, 1),
    block=(THREADS_PER_CTA, 1, 1),
)
```

`src_tma` is the TMA-compatible global tensor. The output tensor is only for debugging: after the TMA load, the kernel copies SMEM back to GMEM to verify.

Minimal TMA Load: Kernel Side

SMEM tile and mbarrier

```
@cute.kernel
def tma_load_kernel(tma_atom,
                    src_tma,
                    out,
                    smem_layout):
    block_m, block_k, _ = cute.arch.block_idx()

    smem = SmemAllocator()
    smem_tile = smem.allocate_tensor(
        cutlass.BFloat16,
        smem_layout,
        byte_alignment=1024)
    tma_barrier = smem.allocate_array(
        cutlass.Int64, 1, byte_alignment=8)

    with cute.arch.elect_one():
        cute.arch.mbarrier_init(tma_barrier, 1)
        cute.arch.mbarrier_expect_tx(
            tma_barrier, TILE_M * TILE_K * 2)
```

Partition and issue TMA

```
cute.arch.mbarrier_init_fence()
cute.arch.barrier()

src_tiles = cute.local_tile(
    src_tma, (TILE_M, TILE_K), (None, None))

smem_view, gmem_view = cpasync.tma_partition(
    tma_atom,
    0,
    cute.make_layout(1),
    cute.group_modes(smem_tile, 0, 2),
    cute.group_modes(src_tiles, 0, 2),
)

cute.copy(
    tma_atom,
    gmem_view[(None, block_m, block_k)],
    smem_view,
    tma_bar_ptr=tma_barrier,
)

with cute.arch.elect_one():
    cute.arch.mbarrier_arrive(tma_barrier)
    cute.arch.mbarrier_wait(tma_barrier, 0)
```

This is the core TMA pattern: build views, issue `cute.copy`, then wait for the async copy to complete.